

DESIGN AND ANALYSIS OF ALGORITHMS

DAY 2 – ALGORITHM PROBLEM SOLVING

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

TABLE OF CONTENTS

Experiment 1: First Palindromic String

Experiment 2: Count Common Elements

Experiment 3: Sum of Squares of Distinct Counts

Experiment 4: Equal Value Pairs with Divisibility Condition

Experiment 5: Maximum Element with Least Time Complexity

Experiment 6: Sort and Find Maximum Element

Experiment 7: Unique Elements

Experiment 8: Bubble Sort

Experiment 9: Binary Search

Experiment 10: Sorting Without Built-in Functions

Experiment 1: First Palindromic String

Objective: To find the first palindromic string in an array of strings.

Problem Statement: Given an array of strings, return the first palindromic string. If there is no such string, return an empty string "".

Sample Input: words = ["abc", "car", "ada", "racecar", "cool"]

Sample Output: "ada"

Time Complexity: $O(n \times m)$, where n is the number of strings and m is the average string length.

Explanation: Each word is checked from left to right. A word is palindromic if it reads the same forward and backward. The first matching word is returned.

Python Program

```
def first_palindrome(words):  
    for word in words:  
        if word == word[::-1]:  
            return word  
    return ""  
  
words = input("Enter words separated by spaces: ").split()  
print("First palindromic string:", first_palindrome(words))
```

Execution / Output

```
Enter words separated by spaces: abc car ada racecar cool  
First palindromic string: ada
```

Result: The program was implemented and the expected result was obtained.

Experiment 2: Count Common Elements

Objective: To count the elements in each array that also exist in the other array.

Problem Statement: Given two integer arrays, calculate answer1 as the number of indices i such that $\text{nums1}[i]$ exists in nums2 , and answer2 similarly for nums2 .

Sample Input: $\text{nums1} = [2,3,2]$, $\text{nums2} = [1,2]$

Sample Output: $[2, 1]$

Time Complexity: $O(n + m)$ average time using sets.

Explanation: Sets provide efficient membership testing. Every element of each array is checked against the set containing the values of the other array.

Python Program

```
def count_common(nums1, nums2):
    set1 = set(nums1)
    set2 = set(nums2)

    answer1 = sum(1 for x in nums1 if x in set2)
    answer2 = sum(1 for x in nums2 if x in set1)

    return [answer1, answer2]

nums1 = list(map(int, input("Enter nums1: ").split()))
nums2 = list(map(int, input("Enter nums2: ").split()))

print(count_common(nums1, nums2))
```

Execution / Output

```
Enter nums1: 2 3 2
Enter nums2: 1 2
[2, 1]
```

Result: The program was implemented and the expected result was obtained.

Experiment 3: Sum of Squares of Distinct Counts

Objective: To calculate the sum of the squares of distinct counts of all non-empty subarrays.

Problem Statement: Given a 0-indexed integer array, find the distinct count of every contiguous non-empty subarray, square each count, and return their sum.

Sample Input: [1,2,1]

Sample Output: 15

Time Complexity: $O(n^2)$ time and $O(n)$ auxiliary space.

Explanation: For each starting position, the subarray is extended one element at a time. A set maintains the distinct values and the square of the current count is added to the total.

Python Program

```
def distinct_count_square_sum(nums):
    total = 0

    for i in range(len(nums)):
        distinct = set()
        for j in range(i, len(nums)):
            distinct.add(nums[j])
            total += len(distinct) ** 2

    return total

nums = list(map(int, input("Enter elements: ").split()))
print("Sum =", distinct_count_square_sum(nums))
```

Execution / Output

```
Enter elements: 1 2 1
Sum = 15
```

Result: The program was implemented and the expected result was obtained.

Experiment 4: Equal Value Pairs with Divisibility Condition

Objective: To count pairs of equal values whose index product is divisible by k.

Problem Statement: Given nums and k, count pairs (i,j), where $i < j$, $\text{nums}[i] == \text{nums}[j]$, and $(i*j)$ is divisible by k.

Sample Input: nums = [3,1,2,2,2,1,3], k = 2

Sample Output: 4

Time Complexity: $O(n^2)$ time using the direct pair-checking approach.

Explanation: Every possible pair of indices is examined. A pair is counted only when both the value condition and divisibility condition are satisfied.

Python Program

```
def count_pairs(nums, k):
    count = 0

    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1

    return count

nums = list(map(int, input("Enter elements: ").split()))
k = int(input("Enter k: "))

print("Number of valid pairs =", count_pairs(nums, k))
```

Execution / Output

```
Enter elements: 3 1 2 2 2 1 3
Enter k: 2
Number of valid pairs = 4
```

Result: The program was implemented and the expected result was obtained.

Experiment 5: Maximum Element with Least Time Complexity

Objective: To find the maximum element of a list using the least time complexity.

Problem Statement: Write a program for the given test cases that returns the maximum element. An empty list should be handled appropriately.

Sample Input: [1,2,3,4,5]

Sample Output: 5

Time Complexity: $O(n)$ time and $O(1)$ auxiliary space.

Explanation: A single traversal is enough to find the maximum element. Sorting is not required because sorting would take more time than scanning the list once.

Python Program

```
def find_maximum(arr):  
    if not arr:  
        return None  
  
    maximum = arr[0]  
  
    for x in arr[1:]:  
        if x > maximum:  
            maximum = x  
  
    return maximum  
  
arr = list(map(int, input("Enter elements: ").split()))  
  
result = find_maximum(arr)  
  
if result is None:  
    print("List is empty")  
else:  
    print("Maximum element =", result)
```

Execution / Output

```
Enter elements: 1 2 3 4 5  
Maximum element = 5
```

Result: The program was implemented and the expected result was obtained.

Experiment 6: Sort and Find Maximum Element

Objective: To sort a list using an efficient sorting algorithm and then find its maximum element.

Problem Statement: The algorithm first sorts the list using an efficient sorting algorithm and then finds the maximum element in the sorted list.

Sample Input: [5,2,9,1,7]

Sample Output: Sorted list = [1,2,5,7,9]; Maximum element = 9

Time Complexity: $O(n \log n)$ time using merge sort.

Explanation: Merge sort is used to achieve guaranteed $O(n \log n)$ sorting time. Once sorted, the last element is the maximum. Empty and single-element lists are also handled.

Python Program

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = list(map(int, input("Enter elements: ").split()))
if not arr:
    print("List is empty")
else:
    sorted_arr = merge_sort(arr)
    print("Sorted list =", sorted_arr)
    print("Maximum element =", sorted_arr[-1])
```

Execution / Output

```
Enter elements: 5 2 9 1 7
Sorted list = [1, 2, 5, 7, 9]
Maximum element = 9
```

Result: The program was implemented and the expected result was obtained.

Experiment 7: Unique Elements

Objective: To create a new list containing only the unique elements from the original list and analyze space complexity.

Problem Statement: Given a list of n numbers, create a new list containing each unique element only once.

Sample Input: [3,7,3,5,2,5,9,2]

Sample Output: [3,7,5,2,9]

Time Complexity: $O(n)$ average time and $O(n)$ auxiliary space.

Explanation: A set stores elements that have already appeared. When an element is seen for the first time, it is added to the result list. The additional space is $O(n)$ in the worst case.

Python Program

```
def unique_elements(arr):  
    seen = set()  
    result = []  
  
    for x in arr:  
        if x not in seen:  
            seen.add(x)  
            result.append(x)  
  
    return result  
  
arr = list(map(int, input("Enter elements: ").split()))  
print("Unique elements =", unique_elements(arr))  
print("Space complexity:  $O(n)$ ")
```

Execution / Output

```
Enter elements: 3 7 3 5 2 5 9 2  
Unique elements = [3, 7, 5, 2, 9]  
Space complexity:  $O(n)$ 
```

Result: The program was implemented and the expected result was obtained.

Experiment 8: Bubble Sort

Objective: To sort an array of integers using the bubble sort technique.

Problem Statement: Sort an array of integers in ascending order using bubble sort and analyze its time complexity using Big-O notation.

Sample Input: [5,1,4,2,8]

Sample Output: [1,2,4,5,8]

Time Complexity: Best Case: $O(n)$ with early stopping; Worst/Average Case: $O(n^2)$.

Explanation: Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. The early-stop condition makes the best case linear for an already sorted array.

Python Program

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

arr = list(map(int, input("Enter elements: ").split()))
bubble_sort(arr)
print("Sorted array =", arr)
```

Execution / Output

```
Enter elements: 5 1 4 2 8
Sorted array = [1, 2, 4, 5, 8]
```

Result: The program was implemented and the expected result was obtained.

Experiment 9: Binary Search

Objective: To check whether a given number exists in a sorted array using binary search.

Problem Statement: Given a sorted array arr and a key x, determine whether x exists using binary search and analyze its time complexity.

Sample Input: [3,4,6,-9,10,8,9,30] with KEY = 10

Sample Output: Element 10 is found at position 5

Time Complexity: $O(\log n)$ time and $O(1)$ auxiliary space for iterative binary search.

Explanation: Binary search repeatedly divides the search interval into two halves. The supplied example is not sorted, so the program sorts the input first before applying binary search. Position is reported using 1-based indexing.

Python Program

```
def binary_search(arr, key):
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter key: "))
arr.sort()
pos = binary_search(arr, key)

if pos != -1:
    print("Sorted array =", arr)
    print(f"Element {key} is found at position {pos + 1}")
else:
    print("Sorted array =", arr)
    print(f"Element {key} is not found")
```

Execution / Output

```
Enter elements: 3 4 6 -9 10 8 9 30
Enter key: 10
Sorted array = [-9, 3, 4, 6, 8, 9, 10, 30]
Element 10 is found at position 7
```

Result: The program was implemented and the expected result was obtained.

Experiment 10: Sorting Without Built-in Functions

Objective: To sort an array of integers in ascending order without using built-in sorting functions.

Problem Statement: Sort the array in $O(n \log n)$ time without using built-in sorting functions, using an efficient divide-and-conquer algorithm.

Sample Input: [5,2,8,1,3]

Sample Output: [1,2,3,5,8]

Time Complexity: $O(n \log n)$ time and $O(n)$ auxiliary space with merge sort.

Explanation: Merge sort divides the input into smaller subarrays, recursively sorts them, and merges them. It satisfies the required $O(n \log n)$ time without using a built-in sorting function.

Python Program

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    merged = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

arr = list(map(int, input("Enter elements: ").split()))
print("Sorted array =", merge_sort(arr))
```

Execution / Output

```
Enter elements: 5 2 8 1 3
Sorted array = [1, 2, 3, 5, 8]
```

Result: The program was implemented and the expected result was obtained.